Prof. Joseph G Vella

Department of Computer Information Systems

FICT

Joseph.g.vella@um.edu.mt

Lab Work Submission Cover Sheet

Semester 2 – 2024/2025

Level: First Year (UG)

Group Name/Details: Francesca Nardelli

Mirea Sacco

Surname	First name	Email (@um.edu.mt)	ID	UNIT Code
Nardelli	Francesca	francesca.nardelli.24@um.edu.mt	0166706L	IDB (5) CIS1043
Sacco	Mirea	mirea.sacco.24@um.edu.mt	0173606L	IDB (5) CIS1043

This is our own work.

Lab 1

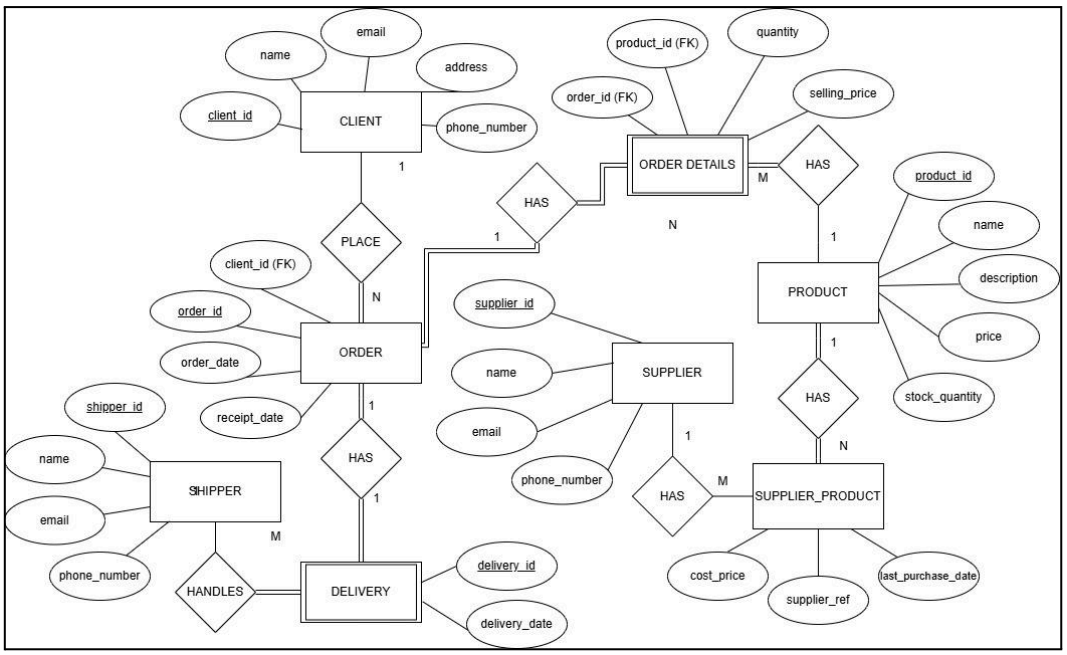
Question A

1. **Identify and draw the main look-up entities (e.g. client)**
   Add some sensible attributes that an order-processing system needs to function

Entity	**Attributes**
Client	client_id (PK) name email phone_number address
Product	product_id (PK) name description price stock_quantity
Supplier	supplier_id (PK) name email phone_number
Shipper	shipper_id (PK) name email phone_number
Order	order_id (PK) client_id (FK) order_date receipt_date
Order_Details	order_id [FK] product_id [FK]

	quantity selling_price
Delivery	delivery_id delivery_date

2. **Build the relationship between entities product and supplier.**
   1 Product can have multiple Suppliers (M:N, using Supplier_Product)
3. **Introduce the entities for order and order details and their relationship. Relate order and order details entities with the other look-up entities (e.g. client and product).**
   1 Order can have multiple Products (M:N, using Order_Details)
   1 client can have multiple Orders (1:N)
4. **Build a relationship between client and order.**
   1 client can place multiple Orders (1:M)
5. **Build a relationship between order and delivery note.**
   1 Order has 1 Delivery (1:1)
6. **Build a relationship between delivery note and shipper.**
   1 Shipper can handle multiple Deliveries (1:M)
7. **Re-check your design by considering the following:**
   (a) Are attributes in place, and whether composite or multiple options are applicable.
   (b) Is there a primary key in each entity?
   (c) Are all relationships properly structured (e.g. between two entities)? Are cardinality and participation constraints in place?
   (d) Is there a weak entity (and therefore weak relationship)?



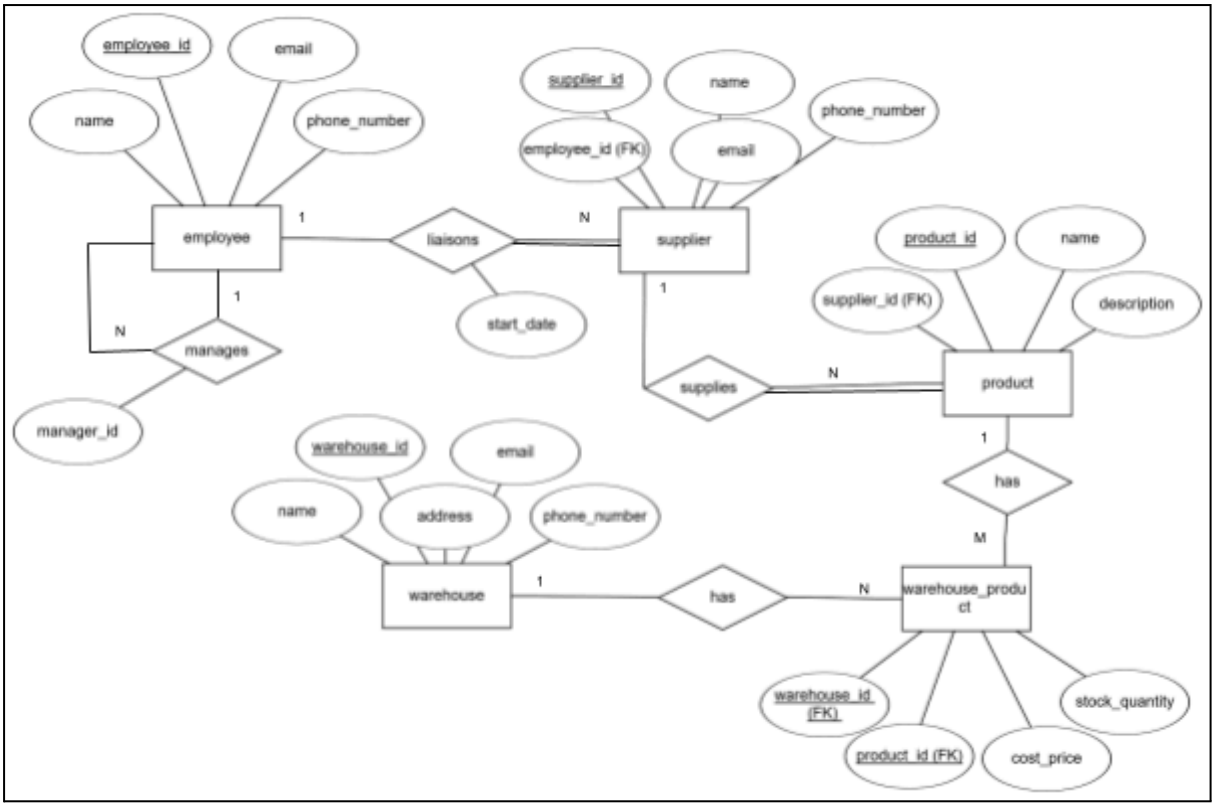
ERM for Question A

Question B

1. Create a schema in your PostgreSQL database to implement the following exercises. During the following work it's best to help yourself with test data. Remember integrity constraints, e.g. not null, will not disrupt processing if the data present and or absence does not fit a constraint (i.e., insert / update / delete commands will give errors).
2. Draw the following ERMs and convert these to relational tables with SQL's Data Definition Language constructs. (Do not use triggers or cascading updates in this exercise).
    - Entities Supplier and Product are related with a 1(p)-N(t) relationship (Supplier supplies these products).
    - Entities Employee and Supplier are related with a 1(p)-N(t) relationship adorned with start-date attribute (Employee liaisons with many suppliers).
    - Entities Product and Warehouse are related with a M(p)-N(p) relationship adorned with items on stock and cost price attributes.
    - An employee can manage many other employees' instances. Submit the reversed engineered ERM and the SQL DDL statements used to implement the database at the end of the session.

For ERM:

Entity	**Attributes**
supplier	supplier_id (PK) employee_id (FK) name email phone_number
product	product_id (PK) supplier_id (FK) name description
employee	employee_id (PK) name email phone_number start_date
warehouse	warehouse_id (PK) name email phone_number address
warehouse_product	warehouse_id (PK) product_id (PK) stock_quantity cost_price
employee_management	manager_id (PK) managed_id (PK)



ERM for Question B

SQL Statements for Question B

```
CREATE TABLE supplier (
   supplier_id SERIAL PRIMARY KEY,
   name VARCHAR(255) NOT NULL,
   email VARCHAR(255) NOT NULL,
phone_number VARCHAR(20)
start_date DATE NOT NULL
);

CREATE TABLE product (
   product_id SERIAL PRIMARY KEY,
   supplier_id INT NOT NULL,
   name VARCHAR(255) NOT NULL,
   description VARCHAR(255),
   CONSTRAINT fk_supplier FOREIGN KEY (supplier_id) REFERENCES
supplier(supplier_id)
);

CREATE TABLE employee (
```

```
    employee_id SERIAL PRIMARY KEY,
    manager_id INT,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    phone_number VARCHAR(20),
);

ALTER TABLE supplier
ADD COLUMN employee_id INT,
ADD CONSTRAINT fk_employee FOREIGN KEY (employee_id) REFERENCES
employee(employee_id);

CREATE TABLE warehouse (
    warehouse_id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    phone_number VARCHAR(20),
    address VARCHAR(255) NOT NULL
);

CREATE TABLE warehouse_product (
    warehouse_id INT,
    product_id INT,
    stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),
    cost_price DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (warehouse_id, product_id),
    CONSTRAINT fk_warehouse FOREIGN KEY (warehouse_id) REFERENCES
warehouse(warehouse_id),
    CONSTRAINT fk_product FOREIGN KEY (product_id) REFERENCES
product(product_id)
);
```

Lab 2

1. SELECT * FROM date.j;

2. SELECT * FROM date.j
   WHERE city = 'LONDON';

3. SELECT DISTINCT CITY FROM date.j;

4. SELECT s,p,j, CAST(qty*0.8 AS INT) AS qty_less_20, qty AS qty, CAST(qty*1.12 AS INT) AS qty_plus_1 FROM date.spj
   WHERE p = 'P1' or p ='P2';

5. SELECT * FROM date.s
   WHERE city = 'PARIS' AND status ='10';

6. SELECT * FROM date.s
   WHERE city = 'LONDON' OR status ='10';

7. SELECT * FROM date.spj
   WHERE qty>=400 AND s = 'S5' AND p = 'P5';

8. SELECT * FROM date.s
   WHERE city = 'PARIS' AND status !='10';

9. SELECT * FROM date.spj
   WHERE p = 'P2' and qty BETWEEN 100 AND 300;

10. SELECT * FROM date.spj
    WHERE qty IN (100, 300, 500, 700, 900);

11. SELECT spj.s, spj.p, spj.j, spj.qty, s.status
    FROM date.spj
    JOIN date.s ON spj.s = s.s
    WHERE s.status = 10;

12. SELECT spj.s, s.sname, spj.p, p.pname, spj.j, spj.qty
    FROM date.spj
    JOIN date.s ON spj.s = s.s
    JOIN date.p ON spj.p = p.p
    ORDER BY s.sname, p.pname;

13. SELECT spj.s, s.sname, spj.p, p.pname, spj.j, j.jname, spj.qty
    FROM date.spj
    JOIN date.s ON spj.s = s.s
    JOIN date.p ON spj.p = p.p
    JOIN date.j ON spj.j = j.j
    WHERE s.status = 20 AND p.colour = 'RED' AND p.city = 'PARIS';

14. SELECT spj.s, spj.p, spj.j, spj.qty*p.weight AS material
    FROM date.spj
    JOIN date.p ON spj.p = p.p
    WHERE spj.j = 'J4';

Lab 3

Part A

1. Project the first, middle, and last names together with their sex and date of birth of all employees.
   qproject := project fname, minit, lname, sex, bdate (employee);

2. Project the first, middle, and last names together with their date of birth for all female employees.
   qfemale := select sex = ‘F’ (employee);
   qproject := project fname, minit, lname, bdate (qfemale);
   or
   qproject := project fname, minit, lname, bdate (select sex = ‘F’ (employee));

3. Select all projects that are located in ‘Stafford’ or are managed by department number 5.
   qselect := select plocation = 'Stafford' or dnum = 5 (projects);

4. Select all male employees whose salary is lower than 35000.
   qselect := select salary < 35000 (select sex = 'M' (employee));

5. Project all first names in employee and dependent relations (i.e. in one relation).
   q1(name) := project fname (employee);
   q2(name) := project dependent_name (dependent);
   qunion := q1 union q2;

6. Show that there are no common first names in employee and dependent relations.
   q1(name) := project fname (employee);
   q2(name) := project dependent_name (dependent);
   qdiff := q1 difference q2;

Then logically, since the number of tuples in qdiff equals the number of fnames in employees, there are no common first names in employee and dependent relations.

7. Project the employee name, ssn, and salary together with the department name each are assigned to.
   qproduct := select dno = dnumber (employee product department);
   qproject := project fname, dname (qproduct);

8. Project the employee name, his supervisor name, and the department name of the supervisor.
   q1(supervisor_name, ssnumber, depnum) := project fname, ssn, dno (employee);
   q2 := select ssnumber = superssn (q1 product employee);
   q4 := select depnum = dnumber (q2 product department);
   q3 := project fname, supervisor_name, dname (q4);

9. Rewrite query 8 with a different order of operations –i.e. but gives the same response.
   q1 := select dno = dnumber (employee product department);
   q2(supervisor_name, ssnumber, sup_dep_name) := project fname, ssn, dname (q1);
   q3 := project fname, supervisor_name, sup_dep_name (select superssn = ssnumber (employee product q2));

10. Assume all locations of interest are in relation dept_location. Which pairs of departments and locations are not found in relation dept_location.
    q1 := project dnumber (department);
    q2 := project dlocation (dept_locations);
    q3 := (q1 product q2);
    q4(dnumber, dlocation) := project dnumber, dlocation (q3);
    q5 := project dnumber, dlocation (q4 difference dept_locations);

11. EXTRA Work: Is there an employee who is younger than his dependent? (Please note that WinRDBI does not compare dates but their ASCII – ignore this and assume date comparison works!)
    q1(employee_name, ssn, employee_bdate) := project fname, ssn, bdate (employee);
    q2 (dependent_name, ssn, dependent_bdate):= project dependent_name, essn, bdate (dependent);
    q3:= (q1 njoin q2);
    q4 := select employee_bdate > dependent_bdate (q3);

12. EXTRA Work: Retrieve the employees who work on all the projects that 'Alicia Zelaya' works on.
    q1(employee_name, ssn) := project fname, ssn (employee);
    q2(project_number, ssn) := project pno, essn (works_on);
    q3 := q1 njoin q2;
    q4:= select employee_name = 'Alicia' (q3);
    q5 := select project_number = 30 or project_number = 10 (q3);

Part B

Question 1

Reduce the following data structure into a relation in 1NF. (Note braces (i.e. {and}) enclose a set of values from a domain). Also create a primary key set for the resultant relation. The first row contains attribute names; you are free to change as required.

StudNum	StudDept	StudA_levels	StudI_Levels
1315	CIS	{Music, Computing}	{Maths, Philosophy}
1344	MATHS	{Maths, Physics}	{Computing}
1399	STATS	{ }	{ }

Relation Student

StudNum	**StudDept**
1315	CIS

1344	MATHS
1399	STATS

Relation Subject

Subject ID	**Subject**
1	Music
2	Computing
3	Maths
4	Philosophy
5	Physics

Relation SubjectStudent

Subject ID	**Student ID**	**Level**
1	1315	A
2	1315	A
3	1315	I
4	1315	I
3	1344	A
5	1344	A
2	1344	I

Primary key → {SubjectID, StudentID}

Question 2

Reduce the following 1NF relation into a 2NF compliant relation. (If need be, do introduce a primary key set). Clearly state and confirm any FDs and how the decomposition progressed.

StudNum	StudName	UnitNum	UnitName	TransGrade
1315	Vella, James	CIS1041	Intro to DB	C+
1315	Vella, James	CIS1099	Prog in C	B+
1344	Desira, Peter	CIS1011	Soft Eng	A
1344	Desira, Peter	CIS1041	Intro to DB	B+
1344	Desira, Peter	CIS1099	Prog in C	A+
1399	Stivala, Belle	CIS1099	Prog in C	A+

StudNum	**StudName**
1315	Vella, James
1344	Desira, Peter
1399	Stivala, Belle

UnitNum	**UnitName**
CIS1041	Intro to DB
CIS1099	Prog in C
CIS1011	Soft Eng

StudNum	**UnitNum**	**TransGrade**
1315	CIS1014	C+
1315	CIS1099	B+
1344	CIS1011	A
1344	CIS1041	B+
1344	CIS1099	A+
1399	CIS1099	A+

Primary Key → {StudNum, UnitNum}

Full Functional Dependency

StudNum → StudName

UnitNum → UnitName

{StudNum, UnitNum} → TransGrade

The partial dependencies were decomposed into full functional dependencies by using 3 relations, Student, Unit, Enrollment, where each attribute is fully functionally dependent on the primary key of its relation.

Question 3

Reduce the following 1NF relation into 3NF compliant relation. (If need be, do introduce a primary key set). Clearly state and confirm any FDs and how the decomposition progressed.

StudNum	StudName	UnitNum	UnitName	TransGrade	GradeDesc
1315	Vella, James	CIS1041	Intro to DB	C+	Average (well almost)
1315	Vella, James	CIS1099	Prog in C	B+	Very Good (really)
1344	Desira, Peter	CIS1011	Soft Eng	A	Excellent (well almost)
1344	Desira, Peter	CIS1041	Intro to DB	B+	Very Good (really)
1344	Desira, Peter	CIS1099	Prog in C	A+	Excellent
1399	Stivala, Belle	CIS1099	Prog in C	A+	Excellent

Relation Student

StudNum	**StudName**
1315	Vella, James
1344	Desira, Peter
1399	Stivala, Belle

Relation Unit

UnitNum	**UnitName**
CIS1041	Intro to DB
CIS1099	Prog in C
CIS1011	Soft Eng

Relations Enrollments

StudNum	**UnitNum**	**TransGrade**
1315	CIS1014	C+
1315	CIS1099	B+
1344	CIS1011	A
1344	CIS1041	B+
1344	CIS1099	A+
1399	CIS1099	A+

Relation Grades

TransGrade	**GradeDesc**
A	Excellent (well almost)
A+	Excellent
B+	Very Good (really)
C+	Average (well almost)

Original Functional Dependencies

StudNum → StudName

UnitNum → UnitName

{StudNum, UnitNum} → TransGrade

{StudNum, UnitNum} → TransGrade → GradeDesc (transitive)

The transitive dependency was decomposed into:

{StudNum, UnitNum} → TransGrade

and

TransGrade → GradeDesc (transitive)

Lab 4

Question 1

Write stored functions that have the following structure (as per slides): i) & ii) Takes no arguments and returns a single integer and a set of records; iii) & iv) Takes an in argument and returns a single integer and a set of records.

i)

The function number_employees() counts the number of records in the emp table.

```
CREATE FUNCTION number_employees()
RETURNS INT AS $$
DECLARE
        dummy INT;
BEGIN
  SELECT COUNT(*) INTO dummy
        FROM scott.emp;
  RETURN dummy;
END;
$$ LANGUAGE plpgsql;
```

ii) The function list_employees() returns all employee records in emp.

```
CREATE OR REPLACE FUNCTION list_employees()
RETURNS SETOF scott.emp AS $$
BEGIN
        RETURN QUERY SELECT * FROM scott.emp;
END
$$ LANGUAGE plpgsql;
```

iii) The function num_salary accepts an integer 'boundary' and returns the number of employees in emp with salary greater than boundary.

```
CREATE FUNCTION num_salary(boundary INT)
RETURNS INT AS $$
DECLARE
        dummy INT;
BEGIN
   SELECT COUNT(*) INTO dummy
        FROM scott.emp
        WHERE sal > boundary;
   RETURN dummy;
END;
$$ LANGUAGE plpgsql;
```

iv) The function emp_salary_check() accepts integer boundary and returns the records of employees in emp where salary is greater than boundary.

```
CREATE OR REPLACE FUNCTION emp_salary_check(boundary INT)
RETURNS SETOF scott.emp AS $$
BEGIN
        RETURN QUERY SELECT * FROM scott.emp
        WHERE sal > boundary;
END
$$ LANGUAGE plpgsql;
```

Question 2

The function salary_check(enum INT) accepts an employee ID and returns their salary.

```
CREATE FUNCTION salary_check(enum INT)
RETURNS INT AS $$
DECLARE
        dummy INT;
BEGIN
   SELECT sal INTO dummy FROM scott.emp
        WHERE empno = enum;
   RETURN dummy;
END;
```

```
$$ LANGUAGE plpgsql;

SELECT ename FROM scott.emp WHERE sal = salary_check(7788);
```

Question 3

Write a function with an 'out' mode argument. Rewrite this function but avoid using an out mode argument.

```
CREATE OR REPLACE FUNCTION dep_employees1(depnum INT, OUT num_of_emps
INTEGER)
AS $$
BEGIN
   SELECT COUNT(*) INTO num_of_emps
         FROM scott.emp where deptno = depnum;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION dep_employees2(depnum INT)
RETURNS INT AS $$
DECLARE num_of_emps INT;
BEGIN
   SELECT COUNT(*) INTO num_of_emps
         FROM scott.emp WHERE deptno = depnum;
         RETURN num_of_emps;
END;
$$ LANGUAGE plpgsql;
```

Question 4

Write a server function that prints 'hello world' on the client (e.g., pgAdmin/SQL/Message window).

```
CREATE OR REPLACE FUNCTION hello_world()
RETURNS void AS $$
BEGIN
         RAISE NOTICE 'hello world';
END;
```

```
$$ LANGUAGE plpgsql;
```

Question 5

Write a function that takes a job type and prints, on the client, the number of employees' having that job. Ensure adequate error control, e.g., job description provided not on file) is coded.

```
CREATE OR REPLACE FUNCTION number_employees(job_type TEXT)
RETURNS void AS $$
DECLARE dummy INT;
BEGIN
	SELECT COUNT(*) INTO dummy
	FROM scott.emp WHERE job = job_type;
IF dummy = 0 THEN
		RAISE EXCEPTION 'No employee found with job description = %',
job_type;
	END IF;
	RAISE NOTICE 'The number of employees with job description % is %',
job_type, dummy;
END;
$$ LANGUAGE plpgsql;
```

Question 6

Reproduce the getFactorial function discussed in class. Change the function's return type to big integer (and rename the function to getBigFactorial); also introduce error capture for i) raise error if the fn argument is less than 1 (return null and raise error); ii) raise error if computation of factorial computation goes out of the big integer range (return null and raise error). Test the function with the following cases:

(ii) Logically, we found that computing the factorial of all fn > 20 will lead to the computation being out of range for BIGINT. Therefore in our IF condition to raise error if computation goes out of integer range, we checked if fn >20.

21! = 51,090,942,171,709,440,000
BIGINT upper bound = 9,223,372,036,854,775,807
21! > BIGINT upper bound

SQL Statements:

```
CREATE OR REPLACE FUNCTION getBigFactorial( fn int)
RETURNS bigint as $$
DECLARE
product bigint;
minus1 int;
BEGIN
	IF (fn > 20) THEN
		RAISE EXCEPTION 'Error: computing factorial; numeric value out of
range';
	END IF;
	IF (fn < 1) THEN
		RAISE EXCEPTION 'Error: Argument less than 1';
	END IF;

	minus1 := fn - 1;

	IF (minus1>0) THEN
		product := fn * getBigFactorial(minus1);
		RETURN product;
	END IF;
	RETURN fn;
END;
$$ LANGUAGE plpgsql;
```

Argument (fn)	Return	Message
3	6	Successfully run. Total query runtime: 47 msec. 1 rows affected.
20	2432902008176640000	Successfully run. Total query runtime: 54 msec. 1 rows affected.
-1		ERROR: Error: Argument less than 1! CONTEXT: PL/pgSQL function scott getfactorial (integer) line 21 at RAISE SQL state: P0001
21		ERROR: ERROR: computing factorial; numeric value out of range CONTEXT: PL/pgSQL function scott.getfactorial(integer) line 18 at RAISE SQL state: P0001

```
1  SELECT getBigFactorial(3);
```

Data Output | Messages | Notifications

	getbigfactorial bigint
1	6

```
1  SELECT getBigFactorial(3);
```

Data Output | Messages | Notifications

Successfully run. Total query runtime: 53 msec.
1 rows affected.

```
1  SELECT getBigFactorial(20);
```

Data Output | Messages | Notifications

	getbigfactorial bigint
1	2432902008176640000

```
1  SELECT getBigFactorial(20);
```

Data Output | Messages | Notifications

Successfully run. Total query runtime: 59 msec.
1 rows affected.

```
1  SELECT getBigFactorial(-1);
```

Data Output | Messages | Notifications

ERROR: Error: Argument less than 1
CONTEXT: PL/pgSQL function getbigfactorial(integer) line 10 at RAISE

```
1  SELECT getBigFactorial(21);
```

Data Output | Messages | Notifications

ERROR: Error: Argument less than 1
CONTEXT: PL/pgSQL function getbigfactorial(integer) line 10 at RAISE

References

- *Documentation: 17: Chapter 41. PL/pgSQL — SQL Procedural Language*. (n.d.). PostgreSQL. Retrieved May 23, 2025, from https://www.postgresql.org/docs/current/plpgsql.html